



**S. B. JAIN INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NAGPUR.**

Post Lab 03

Aim: Detect edges in images with the Canny method and contrast the results with Sobel and Prewitt detectors.

Name of Student:

Roll No.:

Semester/Year:V/III

Academic Session: 2026-2027

Date of Performance:

Date of Submission:

Aim: Detect edges in images with the Canny method and contrast the results with Sobel and Prewitt detectors.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To implement edge detection on grayscale images using the Canny edge detection method.
- To apply Sobel and Prewitt operators for edge detection and understand their working principles.
- To compare and contrast the results obtained from Canny, Sobel, and Prewitt methods in terms of edge clarity, noise sensitivity, and computational complexity.
- To visualize and analyze the differences in edge maps generated by gradient-based (Sobel, Prewitt) and multi-stage (Canny) techniques.
- To develop an understanding of selecting appropriate edge detection methods for different image processing applications.

THEORY:

Understanding Edges in Images

An edge in an image represents a boundary where there is a significant change in intensity or color. Edge detection is crucial for understanding the structure and features within an image, aiding in tasks like object recognition, segmentation, and tracking. Edges are typically detected by identifying areas with high intensity gradients, which can be achieved using various operators that compute derivatives of the image intensity function.

Edges are boundaries between different regions in an image. These are typically caused by:

- Object boundaries
- Surface orientation changes
- Texture changes
- Lighting variations

Basic Concepts of Edge Detection

Edge detection methods can be broadly categorized into two types: gradient-based methods and second-order derivative methods.

1. Gradient-Based Methods

Gradient-based methods detect edges by looking for the maximum and minimum in the first derivative of the image. The gradient of an image measures the change in intensity at a point. The most common gradient-based operators are the Sobel, Prewitt, and Roberts Cross operators.

2. Second-Order Derivative Methods

Second-order derivative methods detect edges by looking for zero crossings in the second derivative of

the image. The Laplacian operator is a widely used second-order derivative method. It highlights regions of rapid intensity change, which correspond to edges.

Common Edge Detection Techniques

1. Sobel Operator

The [Sobel operator](#) computes the gradient of the image intensity at each point, giving the direction of the largest possible increase from light to dark and the rate of change in that direction. It uses two 3x3 convolution kernels to calculate the gradient in the horizontal and vertical directions.

Working of the Sobel Operator

- **Compute Horizontal and Vertical Gradients:** Apply the Sobel kernels to the image to obtain the gradients in the x (horizontal) and y (vertical) directions.

$$\text{Sobel Kernel (Horizontal)} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\text{Sobel Kernel (Vertical)} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

- **Calculate Gradient Magnitude and Direction:** Use the gradients to compute the magnitude and direction of the edge at each pixel.

$$\text{Gradient Magnitude} = \sqrt{G_x^2 + G_y^2}$$

$$\text{Gradient Direction} = \arctan \left(\frac{G_y}{G_x} \right)$$

Where G_x and G_y are the gradients in the horizontal and vertical directions, respectively.

2. Prewitt Operator

Similar to the Sobel operator, the [Prewitt operator](#) calculates the gradient of the image intensity. The main difference is in the convolution kernels used.

Working of the Prewitt Operator

- **Compute Horizontal and Vertical Gradients:** Apply the Prewitt kernels to the image to obtain the gradients in the x and y directions.

$$\text{Prewitt Kernel (Horizontal)} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\text{Prewitt Kernel (Vertical)} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Calculate Gradient Magnitude and Direction:** Use the gradients to compute the magnitude and direction of the edge at each pixel.

$$\text{Gradient Magnitude} = \sqrt{G_x^2 + G_y^2}$$

$$\text{Gradient Direction} = \arctan \left(\frac{G_y}{G_x} \right)$$

3. Roberts Cross Operator

The Roberts Cross operator performs a simple, quick-to-compute 2x2 gradient measurement on an image. It emphasizes regions of high spatial frequency, which often correspond to edges.

Working of the Roberts Cross Operator

- **Compute Diagonal Gradients:** Apply the Roberts kernels to the image to obtain the gradients along the diagonals.

$$\text{Roberts Kernel (Diagonal 1)} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\text{Roberts Kernel (Diagonal 2)} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

- **Calculate Gradient Magnitude and Direction:** Use the gradients to compute the magnitude and direction of the edge at each pixel.

$$\text{Gradient Magnitude} = \sqrt{G_1^2 + G_2^2}$$

$$\text{Gradient Direction} = \arctan \left(\frac{G_2}{G_1} \right)$$

Where G_1 and G_2 are the gradients obtained from the two Roberts kernels.

4. Canny Edge Detector

The [Canny edge detector](#) is a multi-stage algorithm that provides a robust solution to edge detection

problems. It includes the following steps:

- **Gaussian Blur:** Smooth the image to reduce noise using a Gaussian filter.

$$\text{Gaussian Blur Kernel} = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- **Gradient Calculation:** Compute the intensity gradients of the image using methods like Sobel or Prewitt.
- **Non-Maximum Suppression:** Thin out the edges by suppressing all gradients that are not local maxima.
- **Double Threshold:** Identify strong and weak edges based on threshold values.
- **Edge Tracking by Hysteresis:** Finalize the edge detection by connecting weak edges to strong edges if they are part of the same edge segment.

MAIN CODE:

```
import cv2
import matplotlib.pyplot as plt
from google.colab import files
import numpy as np

# Step 2: Upload an image
uploaded = files.upload()

# Step 3: Read the uploaded image
image_path = next(iter(uploaded))
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# Step 4: Apply Gaussian Blur to reduce noise
blurred_image = cv2.GaussianBlur(image, (5, 5), 1.4)

# Step 5: Apply Canny edge detector
edges = cv2.Canny(blurred_image, 50, 150)

# Step 6: Display the original image and the edge-detected image
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.title('Edge Detected Image')
plt.imshow(edges, cmap='gray')
plt.axis('off')
```

```
plt.show()
```

Canny edge detection algorithm is used in computer vision for identifying edges within an image. It helps in highlighting boundaries which are important for tasks like object detection and image segmentation. In this article, we will see how OpenCV's built-in Canny() function detects edges in an image.

Syntax: cv2.Canny(image, T_lower, T_upper, aperture_size, L2Gradient)

Where:

- Image: Input image to which Canny filter will be applied.
- T_lower: Lower threshold value in Hysteresis Thresholding.
- T_upper: Upper threshold value in Hysteresis Thresholding.
- aperture_size: Aperture size of Sobel filter.
- L2Gradient: Boolean parameter used for more precision in calculating Edge Gradient

Return Value: Returns an image with detected edges where edges are marked in white (255) and non-edges in black (0).

1. Using Canny() Function

- cv2.Canny(img, t_lower, t_upper): Applies Canny edge detection algorithm on the image with the given lower and upper threshold values.
- t_lower = 50: Sets lower threshold for edge detection in the Canny algorithm.
- t_upper = 150: Sets upper threshold for edge detection in the Canny algorithm.
- cv2.imshow('original', img): Displays the original image in a window.
- cv2.imshow('edge', edge): Displays the edge-detected image in a window.

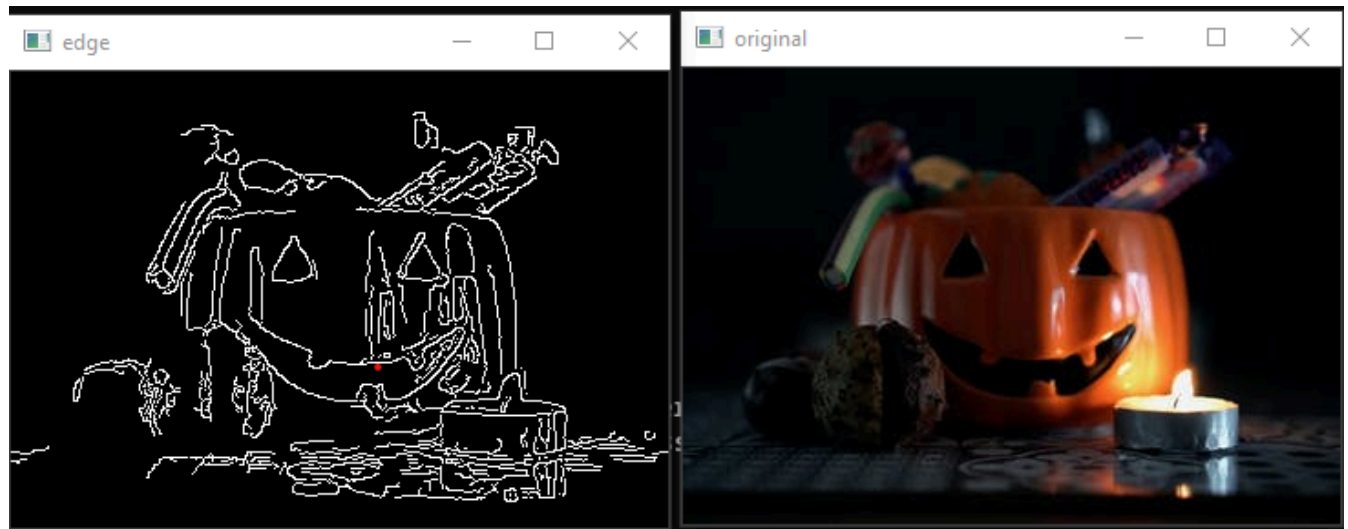
INPUT:



CODE:

```
import cv2
img = cv2.imread("test.jpeg")
t_lower = 50
t_upper = 150
edge = cv2.Canny(img, t_lower, t_upper)
cv2.imshow('original', img)
cv2.imshow('edge', edge)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



2. Canny() function with Aperture_size

This is optional parameter that is used to specify the order of the Sobel filter used to calculate gradient in the Canny algorithm. We can increase Aperture size when we want to detect more detailed features.

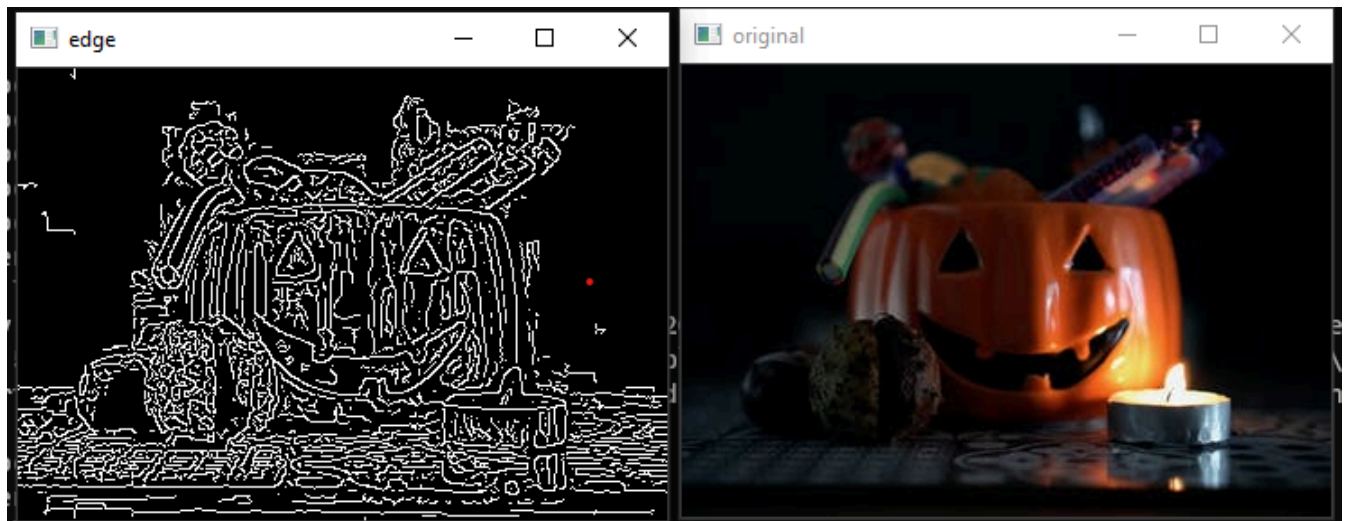
Department of Computer Science and Engineering, S.B.J.I.T.M.R., Nagpur

- `cv2.Canny(img, t_lower, t_upper, apertureSize=aperture_size)`: Applies Canny edge detection with a lower threshold (`t_lower`), upper threshold (`t_upper`) and Sobel filter aperture size (`aperture_size`) to detect edges in the image.
- `aperture_size = 5`: Defines the Sobel kernel size for gradient computation must be an odd number between 3 and 7 and a larger value captures more detailed edges.

CODE:

```
import cv2
img = cv2.imread("test.jpeg")
t_lower = 100
t_upper = 200
aperture_size = 5
edge = cv2.Canny(img, t_lower, t_upper,
                  apertureSize=aperture_size)
cv2.imshow('original', img)
cv2.imshow('edge', edge)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



3. Canny() function with L2Gradient

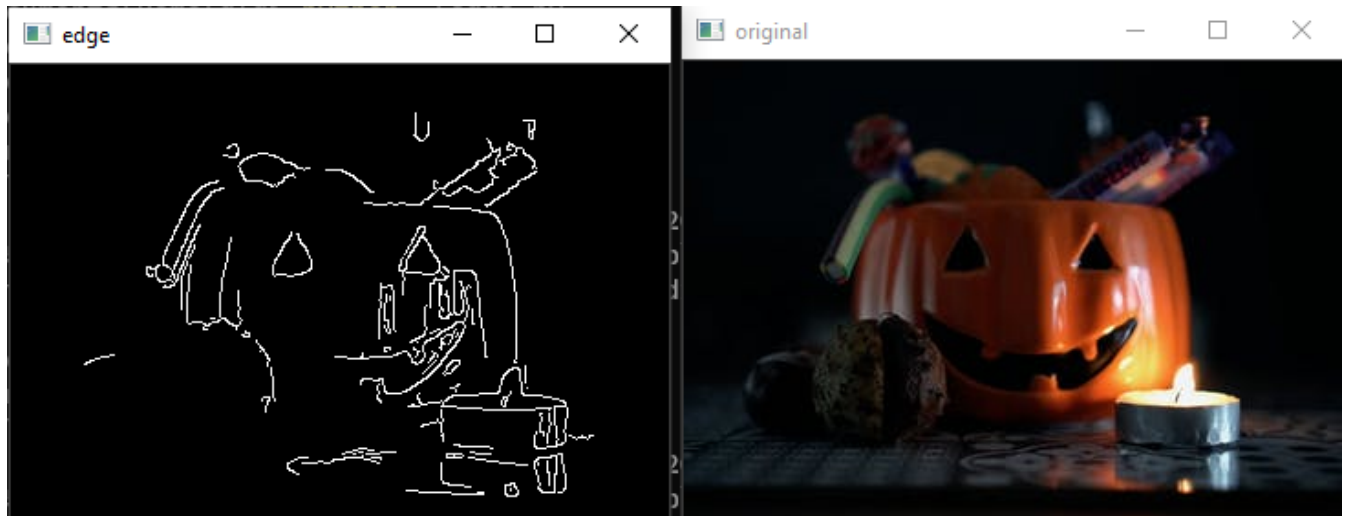
It's a boolean parameter that specifies if we want to calculate usual gradient equation or the L2Gradient algorithm.

- `aperture_size = 5`: Specifies size of the Sobel kernel used to calculate the gradients and we set it to 5 here.
- `L2Gradient = True`: Enables more precise gradient calculation using the L2 norm.
- `edge = cv2.Canny(img, t_lower, t_upper, L2gradient=L2Gradient)`: Applies the Canny edge detection algorithm to the image with the specified thresholds and L2 gradient

CODE:

```
import cv2
img = cv2.imread("test.jpeg")
t_lower = 100
t_upper = 200
aperture_size = 5
L2Gradient = True
edge = cv2.Canny(img, t_lower, t_upper, L2gradient = L2Gradient )
cv2.imshow('original', img)
cv2.imshow('edge', edge)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



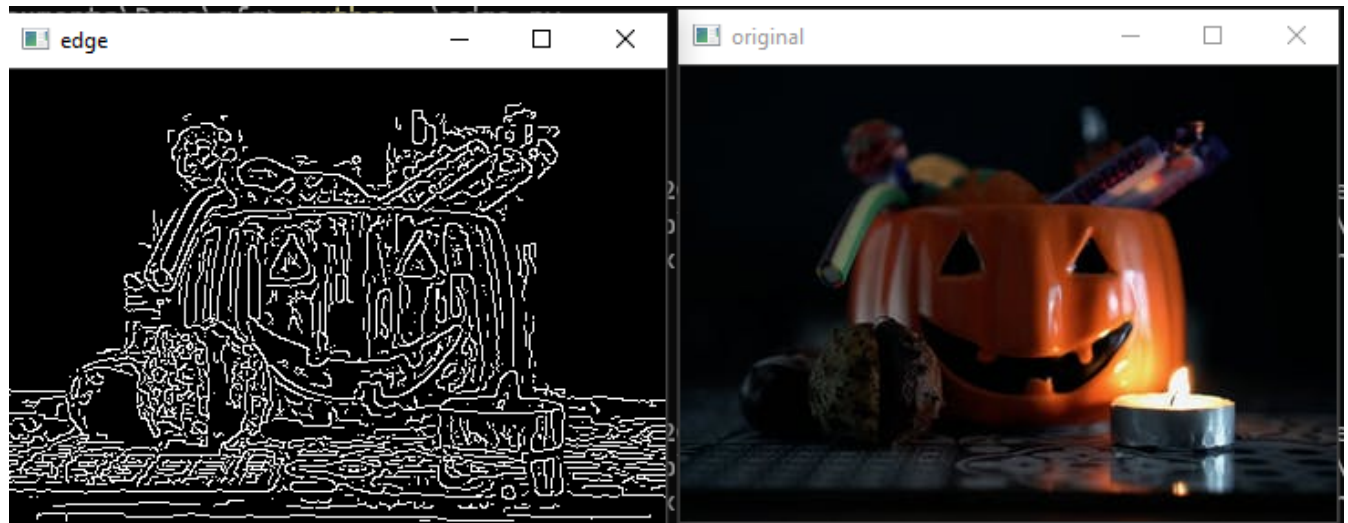
4. Canny() function with both Aperture size and L2gradient

CODE:

```
import cv2
img = cv2.imread("test.jpeg")
t_lower = 100
t_upper = 200
aperture_size = 5
L2Gradient = True
edge = cv2.Canny(img, t_lower, t_upper,
                  apertureSize = aperture_size,
                  L2gradient = L2Gradient )
cv2.imshow('original', img)
cv2.imshow('edge', edge)
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

OUTPUT:



With the flexibility of the Canny() function we can experiment with different parameters to effectively detect edges and enhance the details in our images.

Sobel Edge Detection Code

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# --- Load grayscale image ---
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

if img is None:
    raise ValueError("Image not found. Please check the path and filename.")

# --- Sobel Edge Detection ---
# Sobel in X and Y directions
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

# Compute magnitude
sobel_edges = cv2.magnitude(sobelx, sobely)
sobel_edges = np.uint8(sobel_edges)

# --- Display Results ---
plt.figure(figsize=(10, 4))
```

```
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(sobel_edges, cmap='gray')
plt.title('Sobel Edges')
plt.axis('off')

plt.tight_layout()
plt.show()

# --- Save Result (optional) ---
cv2.imwrite('sobel_edges.jpg', sobel_edges)
```

Prewitt Edge Detection Code

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# --- Load grayscale image ---
img = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)

if img is None:
    raise ValueError("Image not found. Please check the path and filename.")

# --- Prewitt Edge Detection ---
# Prewitt kernels
kernelx = np.array([[ -1, 0, 1],
                    [ -1, 0, 1],
                    [ -1, 0, 1]], dtype=np.float32)
kernely = np.array([[ -1, -1, -1],
                    [ 0, 0, 0],
                    [ 1, 1, 1]], dtype=np.float32)

prewittx = cv2.filter2D(img, cv2.CV_64F, kernelx)
prewitty = cv2.filter2D(img, cv2.CV_64F, kernely)

# Compute magnitude
prewitt_edges = cv2.magnitude(prewittx, prewitty)
```

```
prewitt_edges = np.uint8(prewitt_edges)

# --- Display Results ---
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(prewitt_edges, cmap='gray')
plt.title('Prewitt Edges')
plt.axis('off')

plt.tight_layout()
plt.show()

# --- Save Result (optional) ---
cv2.imwrite('prewitt_edges.jpg', prewitt_edges)
```

Prewitt Operator

The Prewitt operator was developed by Judith M. S. Prewitt. Prewitt operator is used for edge detection in an image. Prewitt operator detects both types of edges, these are:

- Horizontal edges or along the x-axis,
- Vertical Edges or along the y-axis.

Wherever there is a sudden change in pixel intensities, an edge is detected by the mask. Since the edge is defined as the change in pixel intensities, it can be calculated by using differentiation. Prewitt mask is a first-order derivative mask. In the graph representation of Prewitt-mask's result, the edge is represented by the local maxima or local minima.

Both the first and second derivative masks follow these three properties:

- More weight means more edge detection.
- The opposite sign should be present in the mask. (+ and -)
- The Sum of the mask values must be equal to zero.

Prewitt operator provides us two masks one for detecting edges in the horizontal direction and another for detecting edges in a vertical direction.

Prewitt Operator [X-axis] = [-1 0 1; -1 0 1; -1 0 1]

Prewitt Operator [Y-axis] = [-1 -1 -1; 0 0 0; 1 1 1]

Steps:

- Read the image.
- Convert into grayscale if it is colored.
- Convert into the double format.

- Define the mask or filter.
- Detect the edges along X-axis.
- Detect the edges along Y-axis.
- Combine the edges detected along the X and Y axes.
- Display all the images.

Note: All snapshot to be added with your name as caption for all program and try different input images for better understanding.

CONCLUSION:

DISCUSSION QUESTIONS:

- How do the working principles of Canny, Sobel, and Prewitt edge detectors differ?
- Why does the Canny edge detector generally produce thinner and more continuous edges compared to Sobel and Prewitt?
- How does image noise affect the performance of each edge detection method?
- In what situations would Sobel or Prewitt be preferable to Canny for edge detection?
- How does the choice of thresholds in the Canny method influence the detected edges?

REFERENCES:

- <https://opencv.org/blog/edge-detection-using-opencv/>
- <https://www.geeksforgeeks.org/python/python-opencv-canny-function/>
- <https://learnopencv.com/edge-detection-using-opencv/>
- <https://www.geeksforgeeks.org/computer-vision/what-is-edge-detection-in-image-processing/>